

Assignment no.7

Title: Write a program to convert infix expressions to postfix and prefix notation using stacks. Given an infix expression as input, the system should efficiently convert to postfix and prefix notations, considering operator precedence and parentheses.

Name: Diya Ravindra Bhansali

PRN: 125B1F271

Class: SY BTech IT

Source Code:

```
#include <bits/stdc++.h>
using namespace std;
int precedence(char op)
{
    switch (op)
    {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
    }
    return 0;
}
bool isOperator(char ch)
{
    return ch == '+' || ch == '-' || ch == '*' ||
           ch == '/' || ch == '^';
}
```

```

}
string infixToPostfix(string infix)
{
    string postfix;
    stack<char> s;
    for (char ch : infix)
    {
        if (isalnum(ch))
        {
            postfix += ch;
        }
        else if (ch == '(')
        {
            s.push(ch);
        }
        else if (ch == ')')
        {
            while (!s.empty() && s.top() != '(')
            {
                postfix += s.top();
                s.pop();
            }
            if (!s.empty())
                s.pop();
        }
        else if (isOperator(ch))
        {
            while (!s.empty() &&
                s.top() != '(' &&
                precedence(ch) <= precedence(s.top()))
            {
                postfix += s.top();
                s.pop();
            }
            s.push(ch);
        }
    }
}

```

```

    }
    while (!s.empty())
    {
        postfix += s.top();
        s.pop();
    }
    return postfix;
}

int postfixEvaluation(string postfix)
{
    stack<int> s;
    for (char ch : postfix)
    {
        if (isdigit(ch))
        {
            s.push(ch - '0');
        }
        else if (isOperator(ch))
        {
            if (s.size() < 2)
            {
                cout << "Invalid postfix expression!";
                return 0;
            }
            int op2 = s.top();
            s.pop();
            int op1 = s.top();
            s.pop();
            switch (ch)
            {
                case '+':
                    s.push(op1 + op2);
                    break;
                case '-':
                    s.push(op1 - op2);
                    break;
            }
        }
    }
}

```

```

    case '*':
        s.push(op1 * op2);
        break;
    case '/':
        if (op2 == 0)
        {
            cout << "Division by zero!";
            return 0;
        }
        s.push(op1 / op2);
        break;
    case '^':
        s.push(pow(op1, op2));
        break;
    }
}
}
if (s.size() != 1)
{
    cout << "Invalid postfix expression!";
    return 0;
}
return s.top();
}

```

```

string infixToPrefix(string infix)
{
    reverse(infix.begin(), infix.end());
    for (char &ch : infix)
    {
        if (ch == '(')
            ch = ')';
        else if (ch == ')')
            ch = '(';
    }
    string prefix = infixToPostfix(infix);
}

```

```

    reverse(prefix.begin(), prefix.end());
    return prefix;
}

```

```

int main()
{
    string infix, postfix, prefix;
    int ch;
    cout << "Enter Infix Expression: ";
    cin >> infix;
    do
    {
        cout << "\n=====";
        cout << "\n    INFIX EXPRESSION OPERATIONS";
        cout << "\n=====";
        cout << "\n1. Infix to Postfix";
        cout << "\n2. Infix to Prefix";
        cout << "\n3. Postfix Evaluation";
        cout << "\n4. Exit";
        cout << "\n=====";
        cout << "\nEnter your Choice: ";
        cin >> ch;
        switch (ch)
        {
            case 1:
            {
                postfix = infixToPostfix(infix);
                cout << "\nPostfix: " << postfix << endl;
                break;
            }
            case 2:
            {
                prefix = infixToPrefix(infix);
                cout << "\nPrefix: " << prefix << endl;
                break;
            }
        }
    }
}

```

```

    case 3:
    {
        postfix = infixToPostfix(infix);
        cout << "\nPostfix: " << postfix << endl;
        int ans = postfixEvaluation(postfix);
        cout << "Evaluation: " << ans << endl;
        break;
    }
    case 4:
        cout << "\nExiting program...\n";
        break;
    default:
        cout << "\nInvalid Choice!!\n";
    }
} while (ch != 4);
return 0;
}

```

Output:

```
/*
```

Enter Infix Expression: (2+3)*4

```
=====
```

INFIX EXPRESSION OPERATIONS

```
=====
```

1. Infix to Postfix
2. Infix to Prefix
3. Postfix Evaluation
4. Exit

```
=====
```

Enter your Choice: 1

Postfix: 23+4*

```
=====
```

INFIX EXPRESSION OPERATIONS

```
=====
```

1. Infix to Postfix
2. Infix to Prefix
3. Postfix Evaluation
4. Exit

=====

Enter your Choice: 2

Prefix: $*+234$

=====

INFIX EXPRESSION OPERATIONS

=====

1. Infix to Postfix
2. Infix to Prefix
3. Postfix Evaluation
4. Exit

=====

Enter your Choice: 3

Postfix: $23+4*$

Evaluation: 20

=====

INFIX EXPRESSION OPERATIONS

=====

1. Infix to Postfix
2. Infix to Prefix
3. Postfix Evaluation
4. Exit

=====

Enter your Choice: 4

Exiting program...

Process returned 0 (0x0) execution time : 62.910 s

Press any key to continue.

*/

Code Analysis Report

Infix to Postfix and Prefix Conversion using Stacks (C++)

1. Overall Purpose

The program implements the classic Shunting-Yard algorithm (via stack-based operator precedence parsing) to:

- Convert an infix expression → postfix (Reverse Polish Notation)
- Convert an infix expression → prefix (Polish Notation)
- Evaluate a postfix expression

It uses a menu-driven loop so the user can perform any of these operations repeatedly on the same input expression.

2. Core Building Blocks

`precedence(char op)`

Assigns numeric precedence ranks to operators:

- `+, -` → 1 (lowest)
- `*, /` → 2
- `^` → 3 (highest)

This ranking determines the order in which operators are popped from the stack during conversion — higher precedence operators bind tighter and get evaluated (or emitted) before lower ones.

`isOperator(char ch)`

A simple predicate that checks membership in `{+, -, *, /, ^}`, used to distinguish operators from operands and parentheses throughout the code.

3. Infix → Postfix (`infixToPostfix`)

This is the heart of the program, implementing the standard operator stack technique:

Symbol read	Action
Operand (alnum)	Append directly to output
(	Push onto stack (marks a scope boundary)
)	Pop and output stack symbols until '(' is found; discard the '('
Operator	Pop and output all stack-top operators with precedence $\geq$ current operator, then push current operator
End of string	Pop all remaining stack operators into output

Why this works: the stack holds operators "waiting" for an operand of higher precedence to finish first. When a new operator arrives with lower or equal precedence to what's on top of the stack, everything of equal/higher precedence must be resolved (popped to output) before the new operator can be pushed — this enforces left-to-right evaluation order for same-precedence operators (implicit left-associativity) and correct nesting for differing precedence.

The '(' acts as a "wall" on the stack — the popping loop for both ')' and operators stops the moment it sees '(', so parenthesized sub-expressions are always fully resolved as a unit before their surrounding context resumes.

Note on associativity: the condition $\text{precedence}(ch) \leq \text{precedence}(s.top())$ pops equal-precedence operators (left-associative behavior), which is correct for $+$ $-$ $$ $/$. However, it applies the same rule to $^$, meaning $^$ is treated as left-associative here (e.g., $2^3^2 \rightarrow 23^2^$, evaluated as $(2^3)^2$), whereas mathematically $^$ is conventionally right-associative ($2^{(3^2)}$). This is a subtle deviation from standard convention.*

4. Infix $\rightarrow$ Prefix (infixToPrefix)

Rather than writing a separate algorithm, this function cleverly reuses infixToPostfix with a well-known trick:

- Reverse the infix string.
- Swap every '(' with ')' and vice versa (since reversing flips the semantic role of parentheses).
- Run the same postfix algorithm on this reversed/swapped string.
- Reverse the result to get the true prefix expression.

This works because prefix and postfix are mirror images of each other for a given expression tree — traversing the reversed tree in "postfix order" corresponds to traversing the original tree in "prefix order," provided grouping symbols are flipped correctly.

Caveat: this mirror trick is only reliable when all operators are binary, and it can subtly break for $^$ given the left-associativity issue above, or for expressions with unary operators (not handled here at all).

5. Postfix Evaluation (postfixEvaluation)

Uses a numeric operand stack:

- Digit $\rightarrow$ push its integer value.
- Operator $\rightarrow$ pop top two operands (op2 then op1, preserving order since postfix is left-to-right), apply the operator, push result back.
- At the end, exactly one value should remain on the stack — that's the final answer.

Includes basic error handling:

- Stack underflow ($s.size() < 2$ before an operator) $\rightarrow$ "Invalid postfix expression!"
- Division by zero
- Leftover stack size $\neq 1$ at the end $\rightarrow$ malformed expression

Limitation: since it uses $ch - '0'$, this only correctly evaluates single-digit operands (multi-digit numbers like 12 would be read character-by-character and break the logic).

6. Driver (main)

A straightforward do-while menu loop:

- Reads the infix expression once (as a whitespace-delimited token via `cin >>`, so no spaces are allowed within the expression).
- Repeatedly offers postfix, prefix, evaluation, or exit, re-invoking the conversion functions each time (slightly redundant since results aren't cached, but harmless for correctness).

7. Dry Run: $(2+3)*4$

Step	Symbol	Stack	Output
1	(	(	—
2	2	(	2
3	+	(+	2
4	3	(+	23
5	)	(pop '+') → empty	23+
6	*	*	23+
7	4	*	23+4
end	—	pop '*'	23+4*

Postfix: $23+4*$ → Prefix: $*+234$ (matches the sample output in the code comments). Evaluating $23+4*$: push 2, push 3, '+' → 5, push 4, '*' → 20. This matches the documented output of 20.

8. Complexity

- Time: $O(n)$ for postfix conversion (each character pushed/popped at most once); prefix adds two $O(n)$ reversals, still $O(n)$ overall.
- Space: $O(n)$ for the operator/operand stack and output string in the worst case (e.g., all operators of increasing precedence, or all parentheses).

9. Key Design Strengths

- Clean separation of concerns (precedence check, conversion, evaluation as independent functions).
- Elegant reuse of postfix logic for prefix conversion instead of duplicating stack logic.
- Basic but real error handling in evaluation (underflow, division by zero).

10. Limitations to Be Aware Of

- Single-character operands only — no multi-digit number or variable-name support.
- No unary operators (unary -, unary +) or functions.
- '^' treated as left-associative, deviating from mathematical convention.
- No input validation for malformed infix expressions (mismatched parentheses, consecutive operators, etc.) during conversion — only the postfix evaluator checks for structural errors.
- `cin >>` infix won't accept spaces in the expression.